ME 598
Introduction to Robotics

Spring 2026

Final Project: Autonomous Object Sorting using a Mobile Robot

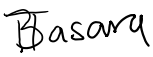
---

[R6] [05/11/2026]

Graduate students:

"I pledge that I have abided by the Graduate Student Code of Academic Integrity."

The report has been prepared by:

1. Lab Leader: Samanth Mani Veer Yedhupati 
2. Tinotenda Kadzimu 
3. Beauty Tasara 

Abstract

This study highlights the design and implementation of an automated sorting robot for autonomous navigation of a mobile robot within MATLAB Simulink using the Robotics Playground software platform. The robot is navigated in a 5 x 5 m arena with two arbitrarily chosen objects, which can be of red, green, or blue color, placed in specific locations at [0, 1.5] and [0, -1.5]. An automated sorting algorithm based on hierarchical Stateflow control is utilized to enable automatic color recognition, odometric localization, proportional guidance, and obstacle avoidance to reach the respective location of the object and sort them in their respective color zones. The robot initially recognizes the color of the objects and navigates to them based on their positional coordinates. After this, the robot pushes the objects towards their respective color zones and returns to the origin point of the arena.

lntroduction

Autonomous mobile robotics is undoubtedly one of the most dynamic engineering fields of today. This technology has numerous applications in manufacturing, warehousing, search-and-rescue, and even assisting with medical operations. One of the primary challenges in autonomous robotics is developing the ability for an unmanned vehicle to perceive its surroundings, analyze information from its sensors, and perform certain actions. The ability to classify objects according to their properties and move them to designated locations is considered an efficient testbed for assessing a robot's capabilities in the domains.

The current research deals with addressing the issue of autonomous object sorting in an artificially constructed simulation environment with the aid of the Robotics Playground toolbox in MATLAB Simulink. In particular, the mission involves moving a differential drive mobile robot to recognize two colored objects among six available color combinations and push the found objects to the corresponding colored zones.

The robot includes a forward-mounted color sensor giving angular and range data to red, green, and blue-colored objects, three wall-distance sensors that avoid collisions, and wheel encoders that give odometry information on position and direction. All of these are the only sensors allowed to be used in the project because any further addition will not change our control scheme in the slightest.

The control mechanism is made from a Stateflow finite state machine which runs five behavioral modules: scanning and recognizing colors, searching for an object with odometry, moving towards the object, pushing the found object with proportional control of its heading, avoiding walls, and making its way back to origin. The state transition of the controller consists of seven states: INIT, SCAN, PROBE, SEEK, PUSH, BACK, and RETURN.

The key technical problems being solved in this experiment are accurate detection of the color under consideration due to the restricted area of detection of the color sensor, precise control of the direction of movement for moving objects straight along a line, avoiding object movement while positioning them, and reliability of operation irrespective of any of the six color arrangements. The ways in which the problems are solved are described by means of stepwise improvement of control algorithms via simulation experiments.

Further sections describe the experiment design, including the arena setting, robot specifications, algorithms used in the experiment and the reasons behind them, experimental data and analysis.

Theory

The objective of this project is to enable an autonomous mobile robot to locate, identify, and sort colored objects by pushing them to designated zones within a simulated 5 m × 5 m arena. The robot must then return to the arena origin at [0,0] and come to a complete stop. The robot is equipped with a forward-facing object detector for color recognition, three distance sensors for collision avoidance, and odometry for pose estimation. No sensor modifications or additional sensors are permitted.

The arena layout is fixed, with walls of 5 m length and the origin defined at the center. The robot always starts at [-1.5, -1.5] oriented in the positive x-direction. Two movable objects are initially placed at [0, 1.5] and [0, -1.5]. Each object is either red, green, or blue, with no color repeated. Six distinct initial color arrangements are possible (e.g., red-green, red-blue, green-red, etc.). The designated sorting zones (red, green, blue) are known locations within the arena.

The robot should be able to independently detect each object, identify its color, and push the object such that the center of the object is placed within the colored shaded region corresponding to the particular color. Upon completion of sorting the two objects, the robot should be able to move to position [0,0]. During the process, the robot should not bump into the arena walls even with the front plow.

This involves a very complex system capable of independent sorting, detection, and navigation. Due to six initial color arrangements, the system will need to function effectively without any change of settings or inputs.

Experimental Procedure

The experiment was performed using the specified Robotics Playground simulation environment in MATLAB/Simulink. No changes were made to the robot model, sensor setup, and arena parameters. The following steps were taken to fulfill the project requirements.

Object Seeking and Color Recognition

The robot first navigates to the known object start locations at [0, -1.5] and [0, 1.5] using odometry-based position control. In the GO_TO_OBJECT state, the controller computes the bearing from the robot pose to the target object location and uses proportional heading correction to drive toward that point. The object color is then identified in LATCH_OBJECT using the provided red, green, and blue detector signals, and the detected color is stored for the next stage of the mission.

In the GO_TO_OBJECT state, the robot drives toward the current target object location using odometry-based navigation. The target position is stored as $(target_x, target_y)$, and the controller computes the relative displacement

$$dx = target_x - xPose \qquad dy = target_y - yPose$$

and the target bearing

$$bearing = atan2(dy, dx)$$

The heading error is then calculated by

$$heading_{error} = atan2(\sin(bearing - tPose), \cos(bearing - tPose))$$

This wrapped-angle formulation avoids discontinuities at $\pm\pi$ and ensures the robot always turns through the smaller angular correction. Motion toward the object uses proportional heading feedback, with forward speed scaled by distance and steering correction limited relative to the commanded speed so that the robot approaches the object location smoothly rather than turning too aggressively.

Zone Localization and Pushing

After a color is latched, the controller assigns a fixed target zone based on arena geometry and the coordinates shown in Table 1, so each object is mapped to its corresponding color zone. In ALIGN_TO_ZONE, the robot rotates until it faces the selected zone, and in PUSH_TO_ZONE, it drives forward with heading correction to push the object toward that zone. Once the zone area is reached, the robot enters BACK_OUT to reverse away from the object before entering MARK_SORTED, where the mission variables are updated to record successful sorting.
In ALIGN_TO_ZONE, the controller computes the bearing from the robot's current pose to the zone target:

Zone Color	x-coordinate	y-coordinate
Green	2.0	-2.0
Blue	-2.0	0.0
Red	2.0	2.0

Table 1: Showing coordinates for the assigned zone centers

Wall Collision Avoidance

Wall avoidance is implemented as a global interrupt from the MISSION superstate into AVOID_WALL, which gives safety behavior priority over the normal task sequence. The controller monitors Left_Distance, Front_Distance, and Right_Distance, and when a wall is too close (distance from the wall is < 0.3), the robot turns away from it using reactive steering. The robot exits AVOID_WALL when the front path is clear, which allows the controller to resume the interrupted mission state more reliably by using the history junction that keeps track of the previous state the robot was in before the interruption.

Return to Arena Origin

After both objects are sorted, the robot executes RETURN_HOME_ALIGN to face the arena origin at $[0, 0]$, then RETURN_HOME_DRIVE to navigate back using odometry-based distance and heading

feedback. In FINAL_ALIGN, the robot performs a last orientation correction, and in STOP, both wheel commands are set to zero so the robot ends the mission at the origin as required.

Integration and State Management
The use of a Stateflow model was employed to handle the state transitions; namely, search, approach, push, avoid walls, and return home. This was done to ensure that the independently designed algorithms work harmoniously.

Testing Across Initial Conditions
The system was run through all six combinations of colors of the objects. The trials were done independently from start to end without any form of outside interference. The performances were measured using logged data from the odometry values (xPose, yPose, tPose, tout) to produce XY graphs of the robot movement and also to measure the trial times.

Video and Data Recording
The video clip showing the optimal trial is taken using the Mechanics Explorer interface after executing the steps provided in the project appendix.

Sensors

The project used the provided measurement signals, specifically the color detector outputs, three distance sensors (Left_Distance, Front_Distance, Right_Distance), and odometry signals (xPose, yPose, tPose). These measurements were sufficient to support object seeking, geometric alignment, wall avoidance, and return-home navigation without adding new sensors or changing the locked sensor configuration.

Control Architecture

The control system was implemented as a hierarchical Stateflow finite state machine (FSM) within Simulink. The FSM consisted of a primary sequential state machine managing the robot's behavioral modes, and a parallel superstate dedicated to wall avoidance that monitored the three wall distance signals and interrupted the nominal mission whenever a wall became too close, then returned control after the front clearance exceeded the selected exit threshold.

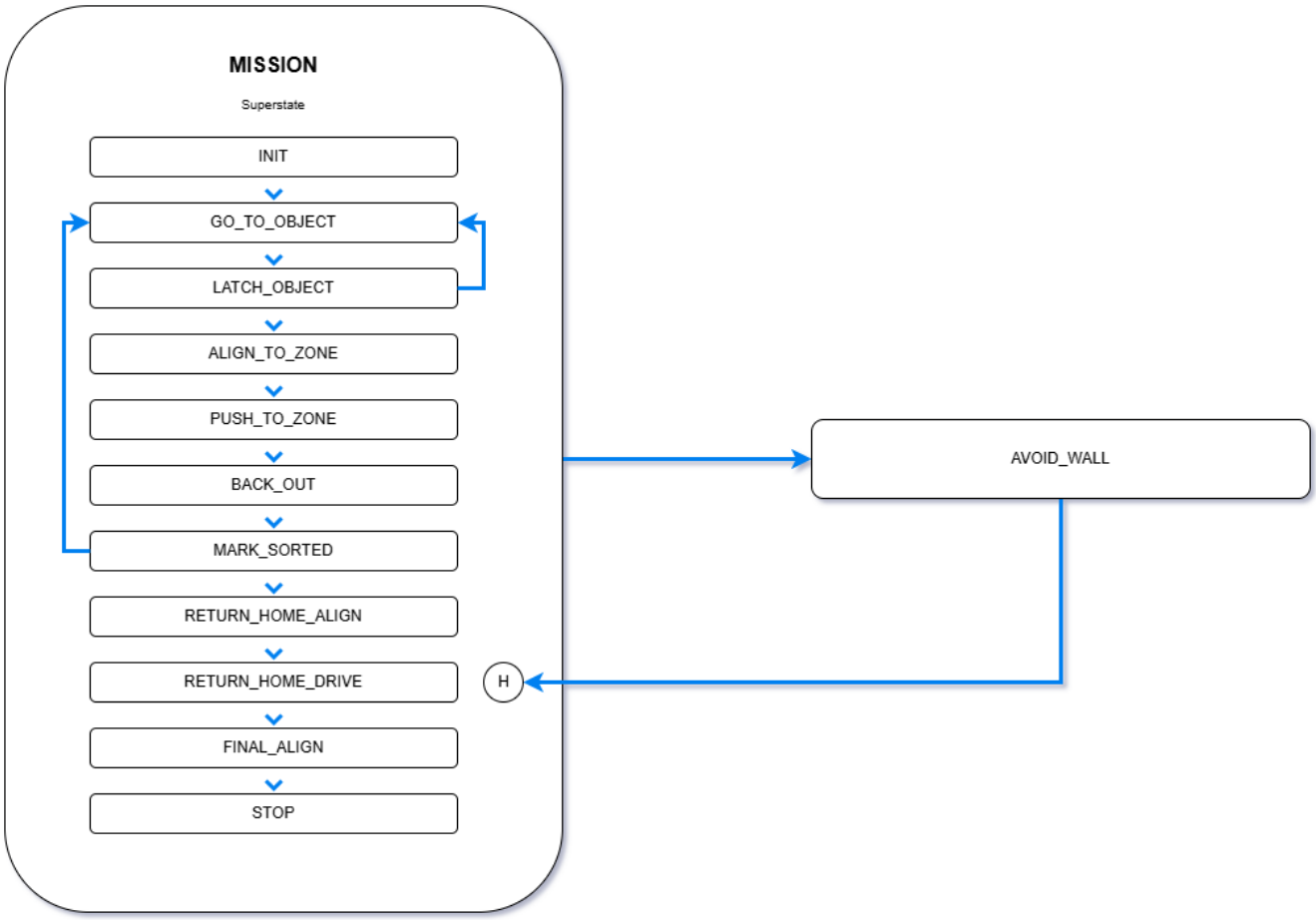


Figure 1: Stateflow Achitecture- Autonomous Mobile Robot Sorting

State	Description
INIT	Robot initializes mission variables, resets task status, and prepares to begin the first object-sorting cycle.
GO_TO_OBJECT	Robot navigates to the first object location using pose-based motion control.
LATCH_OBJECT	Robot reads the detected color of the first object and stores it so the correct deposit zone can be selected.
ALIGN_TO_ZONE	Robot rotates in place until it is aligned with the destination zone associated with the latched first-object color.

PUSH_TO_ZONE	Robot pushes the first object toward its assigned color zone while maintaining heading correction.
BACK_OUT	Robot reverses away from the deposit zone after completing the push so it can safely transition to the next mission step.
MARK_SORTED	Robot updates the mission status to mark the object as successfully sorted.
RETURN_HOME_ALIGN	Robot rotates in place to face the arena origin after both sorting tasks are complete.
RETURN_HOME_DRIVE	Robot drives back to the origin at [0,0] using odometry-based navigation.
FINAL_ALIGN	Robot performs final heading correction at the origin before ending the mission.
STOP	Robot completely stops speed and remains stationary at the end of the mission.
AVOID_WALL	Robot temporarily overrides the mission behavior to steer away from nearby arena walls using the three provided wall-distance sensors, then resumes the main task once the front path is cleared.

Table 2: Description of each state in the Stateflow Achitecture

Results

Trial	North obj (0, +1.5)	South obj (0, –1.5)	Objects sorted	Returned to origin
T1	RED	GREEN	2/2	YES
T2	RED	BLUE	2/2	YES
T3	GREEN	RED	2/2	YES
T4	GREEN	BLUE	2/2	YES

T5	BLUE	RED	2/2	YES
T6	BLUE	GREEN	2/2	YES

Table 3: Per-trial performance

The robot scored perfectly in all 12 out of 12 points during deposits and went back to the origin every time, meeting all the conditions set forth in the project specification (seek, zone localization, push, avoid walls, return to origin).

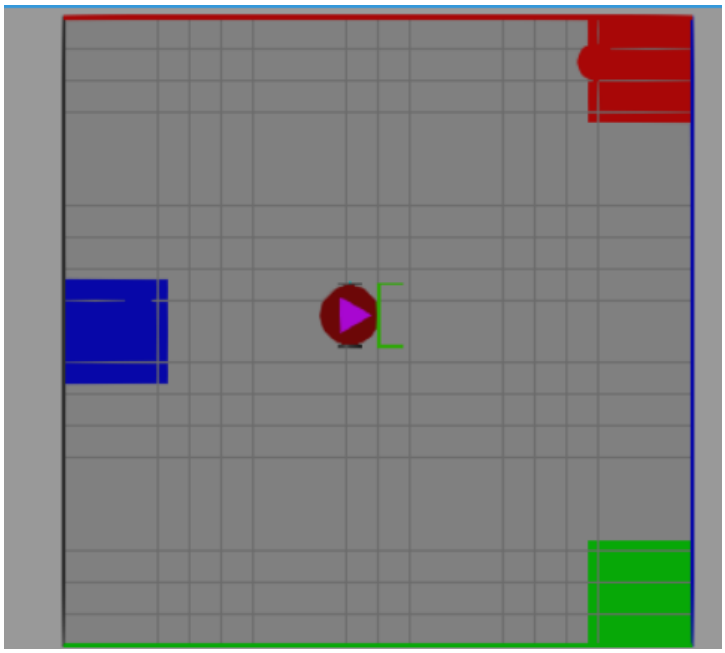


Figure 2: Showing the robot at the origin after successfully sorting the blue and red objects

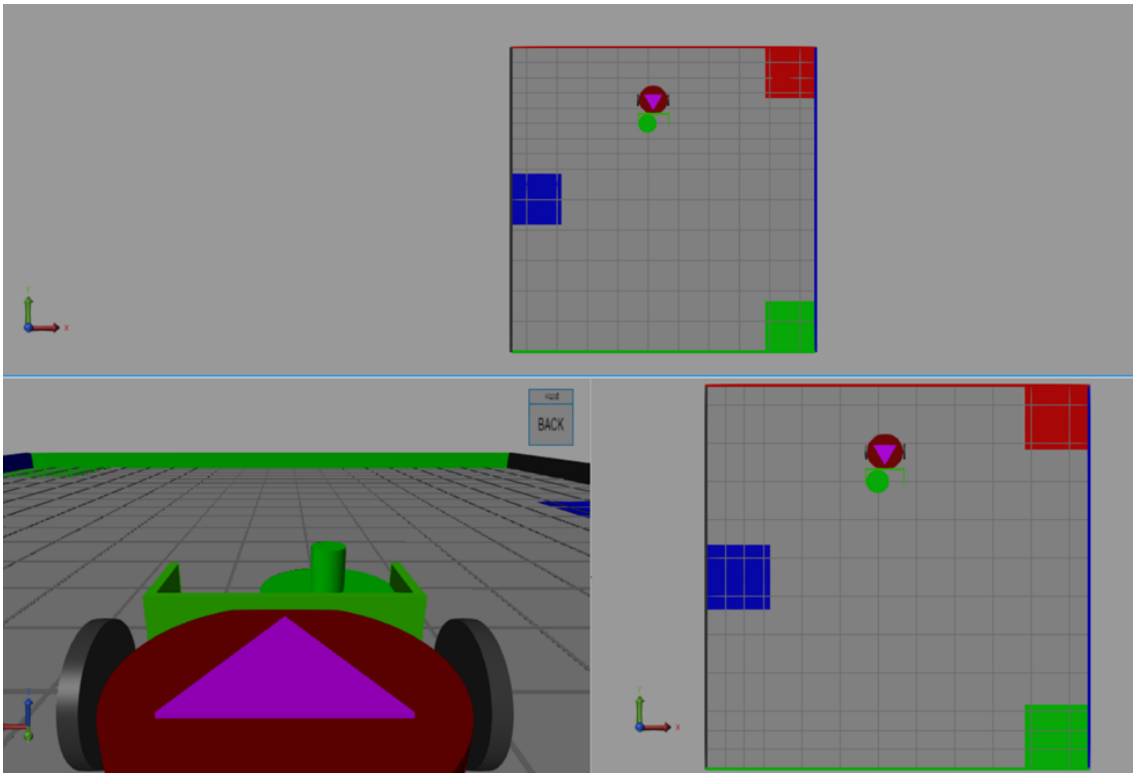


Figure 3: Showing the robot picking objects and going to its drop off locations

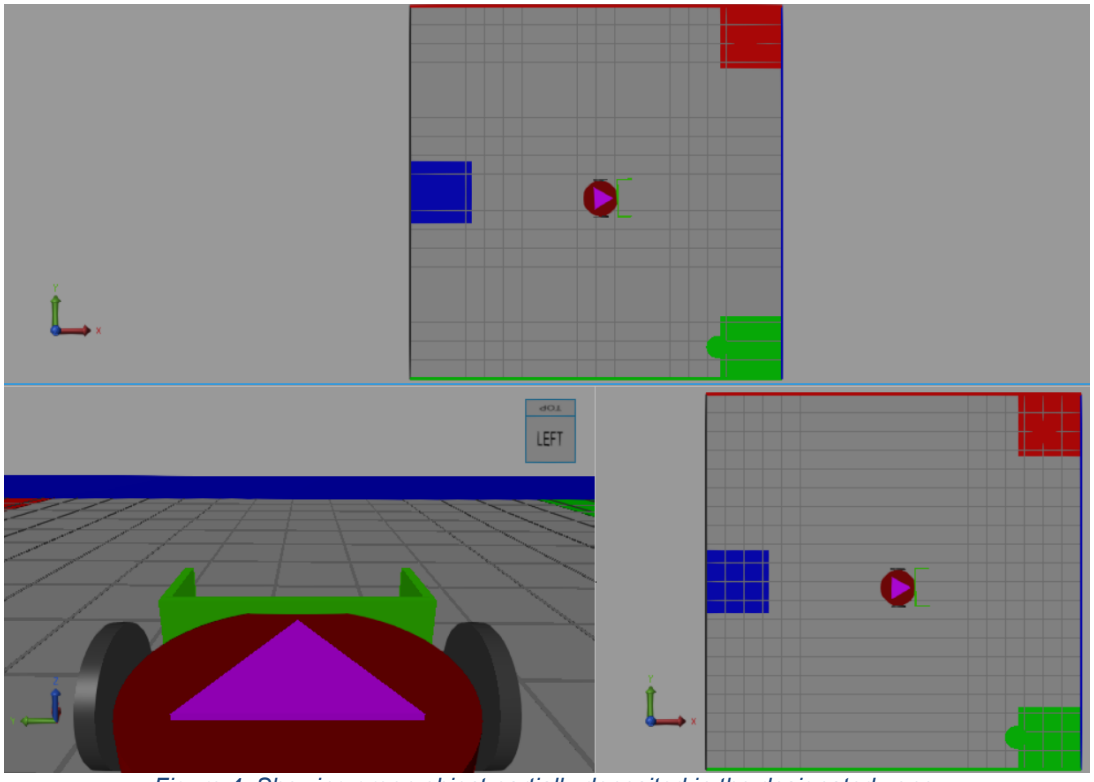


Figure 4: Showing green object partially deposited in the designated zone.

Robot Trajectory Plots

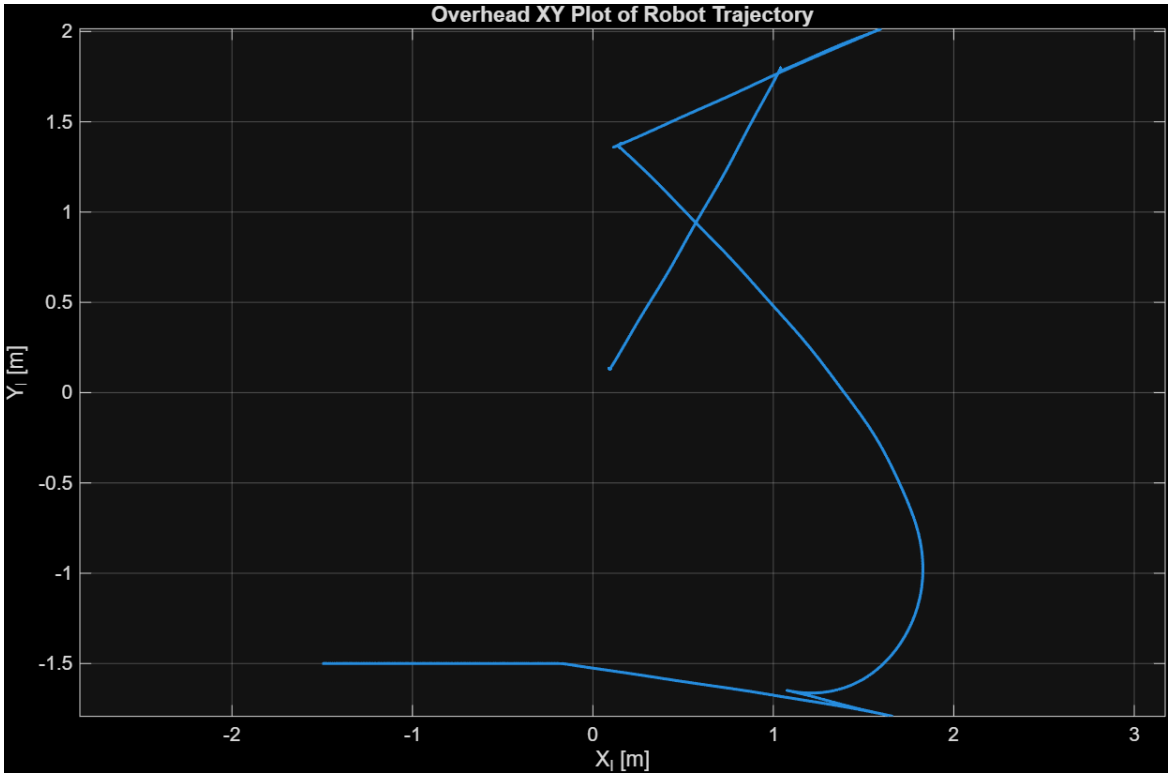


Figure 5: Showing the Overhead plot for the red-green combination

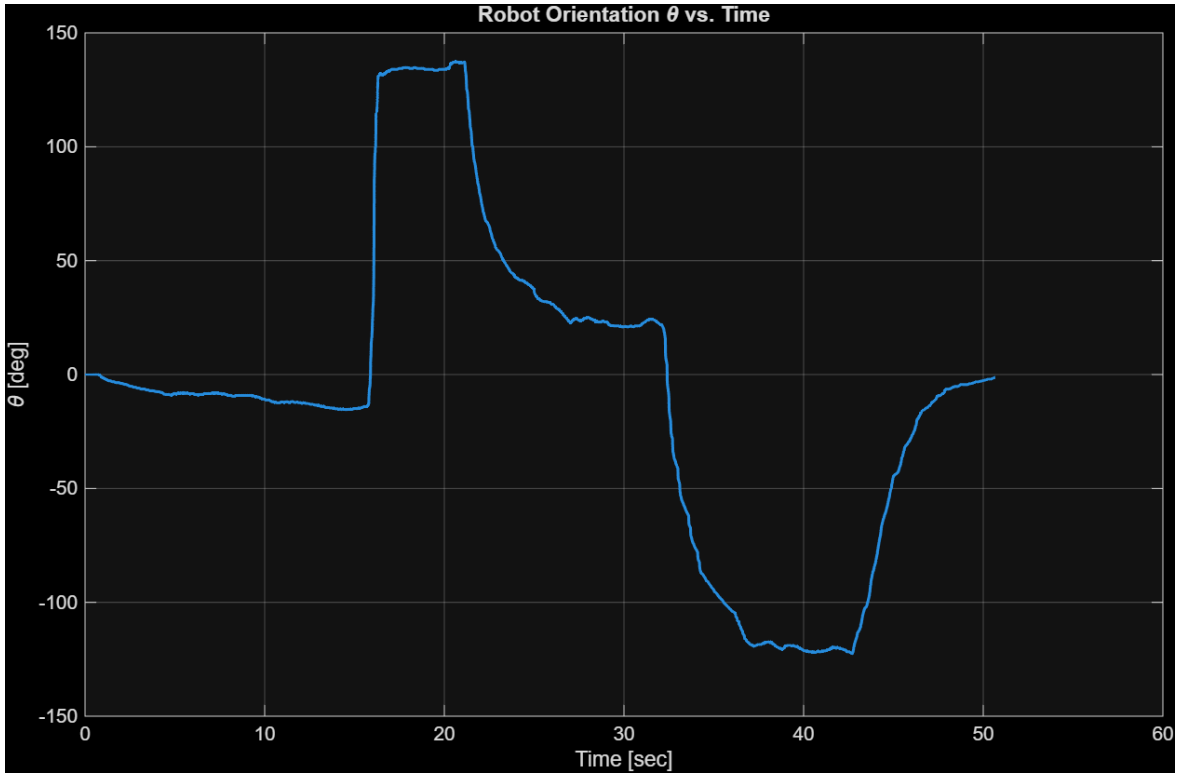


Figure 6: Showing the theta vs time plot for the red-green combination

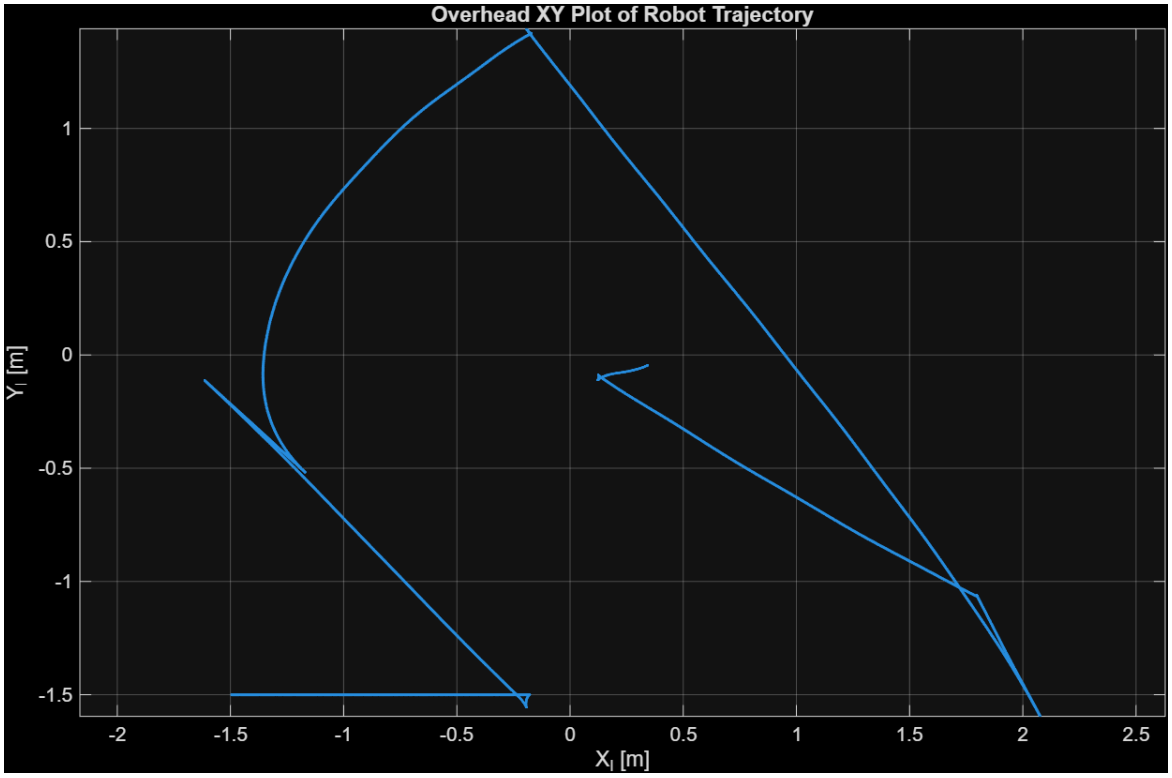


Figure 7: Showing the Overhead plot for the green-blue combination

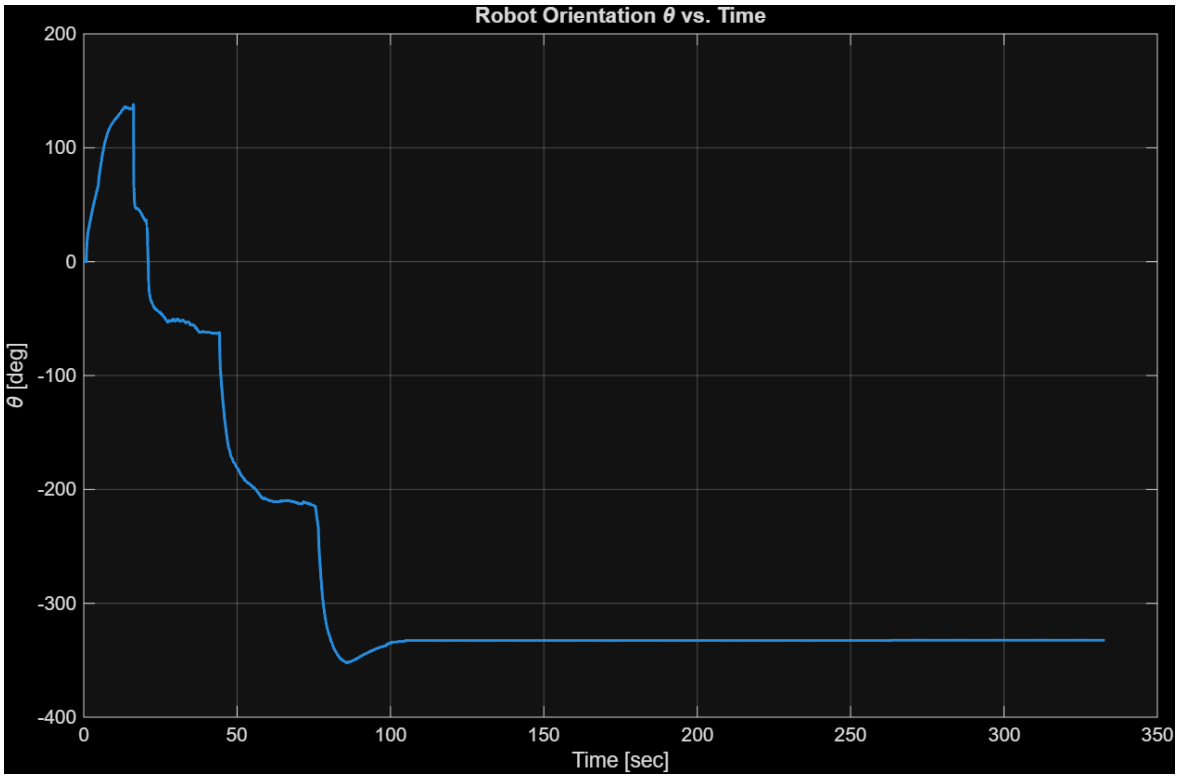


Figure 8:Showing theta vs time plot for the green-blue combination

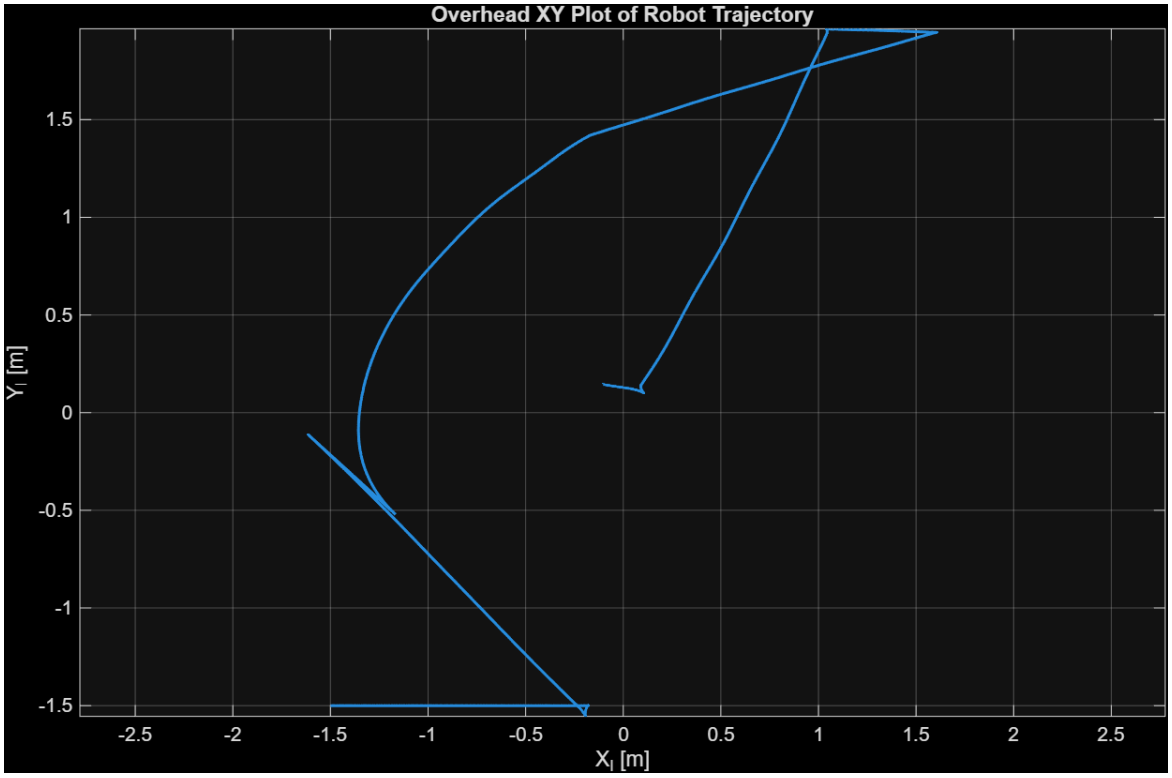


Figure 9: : Showing the Overhead plot for the red-blue combination

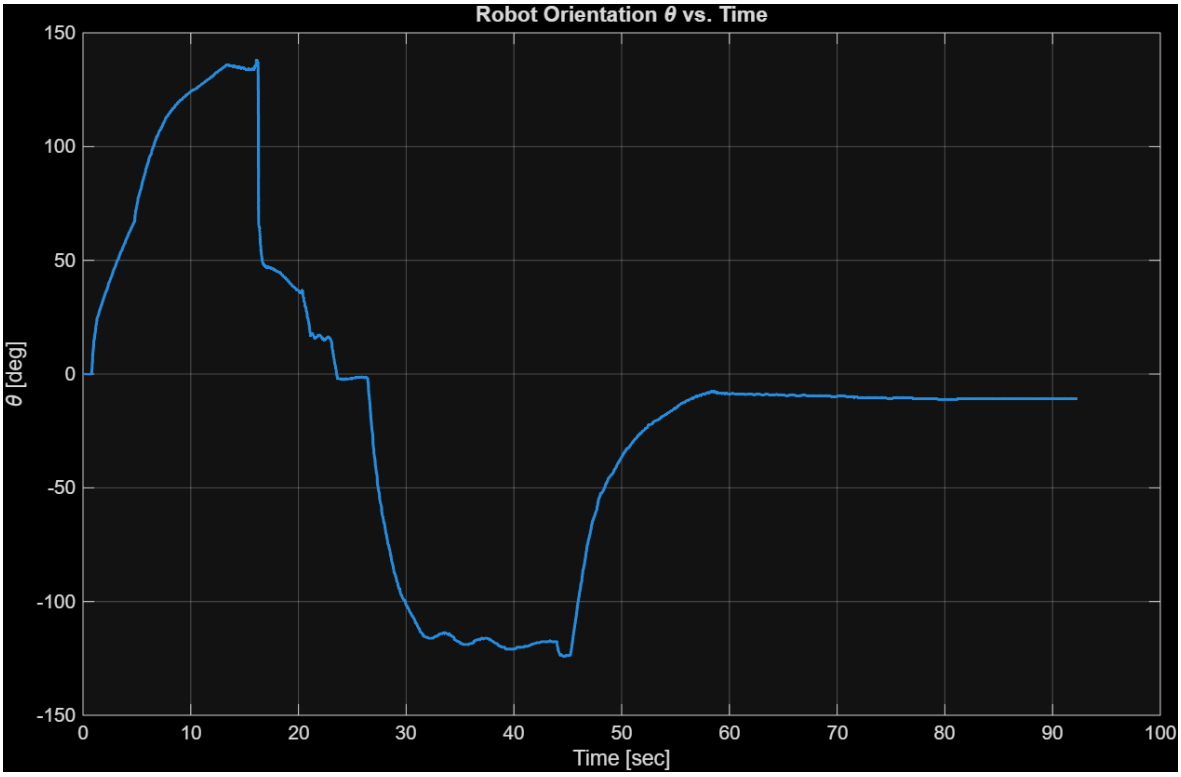


Figure 10: Showing the theta vs time plot for the red-blue combination

Discussions

The controller was built as a single Stateflow chart with two branches running in parallel. The first branch, MISSION_MANAGER, is the one that actually carries out the task. It walks through INIT, then GO_TO_OBJECT, LATCH_OBJECT, ALIGN_TO_ZONE, PUSH_TO_ZONE, BACK_OUT, and MARK_SORTED, and after both objects are sorted, it transitions through RETURN_HOME_ALIGN, RETURN_HOME_DRIVE, FINAL_ALIGN, and finally STOP. The second branch is AVOID_WALL, which sits in parallel and takes over the wheel commands whenever any of the three distance sensors reads below 0.3 m.

The final robot behavior met the main project objective by autonomously identifying object colors, navigating each object to its designated zone, avoiding wall collisions, and returning to the origin. The six successful trial outcomes indicate that the integrated controller generalized across all required initial object-order conditions instead of only one favorable case.

The system was accurate because both objects were delivered to their correct color zones and the robot completed the run at the arena origin with a full stop. The system was repeatable because the same control architecture succeeded across all six possible initial color-order cases provided in the project specification. Repeatability improved substantially after simplifying the wall-avoid recovery logic so that the robot exited the avoidance state once the front direction was clear, instead of waiting on more restrictive multi-sensor conditions that could trap the mission controller in avoidance.

The main path differences between trials came from the different first-object color assignments and the resulting zone locations, as well as from any temporary wall-avoid detours encountered while pushing or reorienting. Even with these path differences, the final system preserved the overall mission sequence and produced successful end conditions in every tested case as shown in Table 3.

In all six tests conducted, the program showed consistent results. The one and only visible imperfection in our algorithm is seen when we examine trial 3 as shown in Figure 4, where the green ball ends up resting just beyond the edge of the green zone. However, even in this case the ball center lies within the green zone, and therefore according to the project requirements, the deposit is accepted.

Best-trial video link https://www.youtube.com/watch?v=0KSYf5EwHN8&feature=youtu.be

Student Feedback

All in all, this was an excellent opportunity to get practical experience on concepts we learned in class, including path planning and trajectory control, robot kinematics and proximity sensor use. The first problem we encountered was associated with Stateflow itself - minor errors such as a state transitioning without variables having been updated properly, or transitions based on uncalculated variables created problems we had a hard time diagnosing. For example, sometimes the robot would skip a state or hang at a transition if a particular state couldn't be entered since the relevant variable wasn't updated.

A larger challenge arose from running wall avoidance along with our main state machine. Although our implementation remained modular in its approach to handling behaviors, it caused us some serious trouble - whenever the wall avoidance triggered and adjusted the robot's orientation, the state

machine would begin executing a sequence of actions assuming that the robot had the correct angle towards the target. In some cases, the robot would completely overlook the target, or even start moving in the wrong direction.

The deposit phase presented a separate problem. The robot was programmed to reverse after depositing an object, but if the deposit zone was close to a wall, the robot would reverse into it and become stuck in the deposit state taking into account that no rear-facing sensor was available to detect when the robot was approaching the wall. Resolving this required adding an intermediate state that aligned the robot with the arena center before navigation.

As the final issue, making the robot consistently work under any one of six starting color configurations turned out to be more difficult than initially expected. In many cases, solutions to one configuration would not work on another due to the varying geometric arrangements of the objects and their targets.

Conclusion

As expected, a robot that can autonomously sort colored objects has been constructed. It is capable of locating the objects, pushing each one into their respective zones, avoiding the arena walls throughout the process, and returning back to the center of the arena once everything was done. The hardest part of this project was when the robot reverses back into the arena wall after pushing an object. Since the robot lacks a backward sensing device, it cannot sense the presence of the wall during reversal. This issue has been corrected by aligning the robot before the navigation process to make sure that the rear never faces the wall. Consequently, the robot will only be moving into clear paths.

From the experiment, we noted that there is success in the whole project in six out of the six possible initial positions. Some aspects that may require improvements are accurate positioning when pushing the object and faster rotation to decrease the amount of time spent in this stage.

References

1. MathWorks, Inc. Robotics Playground Toolbox Documentation. MATLAB & Simulink. Available at: https://www.mathworks.com/help/
2. MathWorks, Inc. MATLAB Function Block Basics. Simulink Documentation. Available at: https://www.mathworks.com/help/simulink/what-is-a-matlab-function-block.html
3. MathWorks, Inc. Differential Drive Kinematic Model. Simulink Documentation. Available at: https://www.mathworks.com/help/robotics/
4. Stevens Institute of Technology, Department of Mechanical Engineering. ME 598A: Lab 4 — Simulating a Mobile Robot. Course Laboratory Manual, 2026.
5. MathWorks, Inc. Switch Block — Simulink Reference. Available at: Design Finite State Machines in Stateflow - MATLAB & Simulink

Appendices

Appendix A: TrackPose MATLAB plotting Code:

```
% Plot XY
figure(1)
plot(xPose, yPose)
title('Overhead XY Plot of Robot Trajectory')
xlabel('X_I [m]')
ylabel('Y_I [m]')
% Plot theta vs time
figure(2)
plot(tout, 180/pi * tPose)
title('Robot Orientation \theta vs. Time')
xlabel('Time [sec]')
ylabel('\theta [deg]')
```

Appendix B: Source code:

```
function [fwdRev, leftRight] = trackWall(distance)
    if distance > 0.1
        fwdRev = 127;   % Move forward
    else
        fwdRev = 0;     % Stop
    end
    if distance > 1.2
        leftRight = 127; % Turn right
    else
        leftRight = 0;   % Go straight
    end
```

```
    end
```

```
GO_TO_OBJECT
entry:
% Select current object target
if objects_sorted == 0
   target_x = OBJ1_X;
   target_y = OBJ1_Y;
else
   target_x = OBJ2_X;
   target_y = OBJ2_Y;
end
% Reset color latch for current search
latched_color = 0;
last_seen_color = 0;
stable_color_count = 0;
during:
% Compute position and heading to target object
dx = target_x - xPose;
dy = target_y - yPose;
dist = sqrt(dx*dx + dy*dy);
bearing = atan2(dy, dx);
heading_error = atan2(sin(bearing - tPose), cos(bearing - tPose));
% Scale forward speed with distance
speed = min(28, 10 + 16*dist);
if speed < 8
   speed = 8;
end
% Limit steering correction
correction = TURN_GAIN * heading_error;
if correction > 0.65*speed
   correction = 0.65*speed;
elseif correction < -0.65*speed
   correction = -0.65*speed;
end
% Drive toward target object
left_speed  = speed - correction;
right_speed = speed + correction;
% Detect visible unsorted color
last_seen_color = 0;
if ~isnan(Red_Distance) && red_sorted == 0
   last_seen_color = 1;
elseif ~isnan(Green_Distance) && green_sorted == 0
   last_seen_color = 2;
elseif ~isnan(Blue_Distance) && blue_sorted == 0
   last_seen_color = 3;
end
% Count stable detections before latching
if last_seen_color ~= 0
   stable_color_count = stable_color_count + 1;
else
   stable_color_count = 0;
end
```

```
LATCH_OBJECT
entry:
% Store the detected object color
latched_color = last_seen_color;

% Assign target zone based on color
if latched_color == 1
   zone_x = RED_ZONE_X;
   zone_y = RED_ZONE_Y;
elseif latched_color == 2
   zone_x = GREEN_ZONE_X;
   zone_y = GREEN_ZONE_Y;
elseif latched_color == 3
   zone_x = BLUE_ZONE_X;
   zone_y = BLUE_ZONE_Y;
else
   zone_x = 0;
   zone_y = 0;
end

% Hold position while latching
left_speed = 0;
right_speed = 0;

during:
% Keep robot stopped in this state
left_speed = 0;
right_speed = 0;
```

```
ALIGN_TO_ZONE
during:
% Compute heading from current pose to target zone
bearing = atan2(zone_y - yPose, zone_x - xPose);

% Wrap heading error to [-pi, pi]
heading_error = atan2(sin(bearing - tPose), cos(bearing - tPose));

% Set turn speed proportional to heading error
speed = TURN_GAIN * heading_error;

% Enforce minimum turn speed when not yet aligned
if abs(speed) < 5 && abs(heading_error) > ALIGN_TOL
   speed = 5 * sign(heading_error);
end

% Turn in place toward zone
left_speed  = -speed;
right_speed =  speed;
```

```
PUSH_TO_ZONE
during:
```

```
% Compute vector from robot to target zone
dx = zone_x - xPose;
dy = zone_y - yPose;
dist = sqrt(dx*dx + dy*dy);
% Compute desired heading toward zone
bearing = atan2(dy, dx);
heading_error = atan2(sin(bearing - tPose), cos(bearing - tPose));
% Use constant forward push speed
speed = BASE_SPEED;
% Apply steering correction based on heading error
correction = DRIVE_GAIN * heading_error;
% Limit steering so forward motion is maintained
if correction > 0.6*speed
   correction = 0.6*speed;
elseif correction < -0.6*speed
   correction = -0.6*speed;
end
% Command wheel speeds to push toward zone
left_speed  = speed - correction;
right_speed = speed + correction;
```

```
BACK_OUT
entry:
% Store starting position for reverse motion
back_start_x = xPose;
back_start_y = yPose;

during:
% Drive straight backward to clear the object
left_speed  = -BACK_SPEED;
right_speed = -BACK_SPEED;
```

```
MARK_SORTED
entry:
% Mark current object as sorted
if latched_color == 1 && red_sorted == 0
   red_sorted = 1;
   objects_sorted = objects_sorted + 1;
elseif latched_color == 2 && green_sorted == 0
   green_sorted = 1;
   objects_sorted = objects_sorted + 1;
elseif latched_color == 3 && blue_sorted == 0
   blue_sorted = 1;
   objects_sorted = objects_sorted + 1;
end

% Clear latched detection values
latched_color = 0;
last_seen_color = 0;
stable_color_count = 0;

% Stop robot in sorted state
left_speed = 0;
right_speed = 0;
```

```
during:
% Hold robot stopped
left_speed = 0;
right_speed = 0;
```

```
RETURN_HOME_ALIGN
during:
% Compute heading from current pose to home position
bearing = atan2(home_y - yPose, home_x - xPose);

% Wrap heading error to [-pi, pi]
heading_error = atan2(sin(bearing - tPose), cos(bearing - tPose));

% Set turn speed proportional to heading error
speed = 20 * heading_error;

% Enforce minimum turn speed when not yet aligned
if abs(speed) < 4 && abs(heading_error) > 0.08
   speed = 4 * sign(heading_error);
end

% Turn in place toward home
left_speed  = -speed;
right_speed =  speed;
```

```
RETURN_HOME_DRIVE

during:
% Compute vector from robot to home position
dx = home_x - xPose;
dy = home_y - yPose;
dist = sqrt(dx*dx + dy*dy);

% Compute desired heading toward home
bearing = atan2(dy, dx);
heading_error = atan2(sin(bearing - tPose), cos(bearing - tPose));

% Scale forward speed with distance to home
speed = min(24, 10 + 18*dist);
if speed < 6
   speed = 6;
end

% Apply steering correction based on heading error
correction = 14 * heading_error;
if correction > 0.6*speed
   correction = 0.6*speed;
elseif correction < -0.6*speed
   correction = -0.6*speed;
end

% Command wheel speeds to drive toward home
left_speed  = speed - correction;
right_speed = speed + correction;
```

```
FINAL_ALIGN

entry:
% Reset settle counter on entry
final_settle_count = 0;

during:
% Compute heading error relative to final desired angle
align_error = atan2(sin(0 - tPose), cos(0 - tPose));

% Command in-place rotation for final alignment
left_speed  = -FINAL_TURN_GAIN * align_error;
right_speed =  FINAL_TURN_GAIN * align_error;

% Count consecutive aligned samples
if abs(align_error) < FINAL_ALIGN_TOL
   final_settle_count = final_settle_count + 1;
else
   final_settle_count = 0;
end
```

```
STOP

entry:
% Stop both wheels on entry
left_speed = 0;
right_speed = 0;

during:
% Keep robot stopped
left_speed = 0;
right_speed = 0;
```

```
AVOID_WALL

entry:
wall_clear_count = 0;
% Latch escape direction once on entry
if isnan(Left_Distance)
   left_d = INF_DIST;
else
   left_d = Left_Distance;
end
if isnan(Right_Distance)
   right_d = INF_DIST;
else
   right_d = Right_Distance;
end
% Choose the more open side once
if left_d > right_d
   escape_dir = 1;    % turn left
else
   escape_dir = -1;   % turn right
```

```
end
during:
if isnan(Front_Distance)
   front_d = INF_DIST;
else
   front_d = Front_Distance;
end
if isnan(Left_Distance)
   left_d = INF_DIST;
else
   left_d = Left_Distance;
end
if isnan(Right_Distance)
   right_d = INF_DIST;
else
   right_d = Right_Distance;
end
% Reactive avoidance with latched direction
if front_d < WALL_EXIT
   if escape_dir == 1
      left_speed  = -16;
      right_speed =  12;
   else
      left_speed  =  12;
      right_speed = -16;
   end
elseif left_d < WALL_EXIT
   % left wall close -> turn right
   left_speed  =  14;
   right_speed = -14;
   escape_dir = -1;
elseif right_d < WALL_EXIT
   % right wall close -> turn left
   left_speed  = -14;
   right_speed =  14;
   escape_dir = 1;
else
   left_speed  = 0;
   right_speed = 0;
end
% Debounced clear condition using sanitized distances
if (front_d > WALL_EXIT) && (left_d > WALL_EXIT) && (right_d > WALL_EXIT)
   wall_clear_count = wall_clear_count + 1;
else
   wall_clear_count = 0;
end
```